

Full Stack Development with MERN Project

Documentation

1. Introduction

- **Project Title:** ShopEZ: E-Commerce Application
- **Team Members:**

Name	Role
Sowmya Kella	Frontend Development, Backend Development, Database Management, Testing

2. Project Overview

- **Purpose:**

ShopEZ is a full-stack e-commerce web application developed using the MERN (MongoDB, Express.js, React.js, and Node.js) stack. The primary objective of this project is to provide a user-friendly online shopping platform where customers can easily browse products, view detailed product information, add items to their cart, and place orders seamlessly.

The project also includes an administrative module that allows administrators to manage product listings by adding, updating, and removing products from the system. Through this application, users can experience a simplified and efficient online shopping process, while administrators can effectively manage inventory and product information.

Additionally, the project demonstrates the implementation of modern web development concepts such as responsive user interfaces, RESTful APIs, database management, user authentication, and client-server communication. It serves as a practical example of full-stack application development using the MERN technology stack and showcases the integration of frontend, backend, and database components into a single functional system.

- **Features:**
 - User Registration and Login
 - Secure Authentication
 - Product Listing and Search
 - Product Details Page
 - Shopping Cart Management
 - Order Placement
 - Admin Dashboard
 - Add, Update, and Delete Products
 - Responsive User Interface

3. Architecture

Frontend Architecture

The frontend of ShopEZ is developed using React.js and serves as the user-facing layer of the application. It provides an interactive and responsive interface that allows users to browse products, manage their shopping cart, place orders, and access account-related functionalities. The frontend is structured using reusable React components, making the application modular and easier to maintain. React Router is used for navigation between pages, while API calls are made to the backend server to fetch and update data dynamically. The responsive design ensures compatibility across desktop, tablet, and mobile devices.

Backend Architecture

The backend is built using Node.js and Express.js, which together handle the business logic and communication between the frontend and the database. The backend exposes RESTful APIs for user authentication, product management, cart operations, and order processing. Express.js manages incoming requests, processes data, performs validations, and returns appropriate responses to the frontend. Middleware functions are used for authentication, authorization, and error handling, ensuring secure and efficient request processing.

Database Architecture

MongoDB is used as the primary database for storing application data. The database follows a document-oriented structure, where data is stored in collections such as Users, Products, and Orders. Mongoose is used as an Object Data Modeling (ODM) library to define schemas and facilitate database operations. User information, product details, and order records are securely stored and retrieved through backend APIs. MongoDB Atlas is used for cloud-based database hosting, providing reliable data storage and accessibility.

System Workflow

The ShopEZ application follows a three-tier architecture consisting of the Presentation Layer (Frontend), Application Layer (Backend), and Data Layer (Database).

1. The user interacts with the React-based frontend.
2. The frontend sends API requests to the Express.js backend.
3. The backend processes the requests and performs the required business logic.
4. MongoDB stores and retrieves application data as needed.
5. The backend sends the processed response back to the frontend.

6. The frontend displays the updated information to the user.

This architecture ensures clear separation of concerns, easier maintenance, improved scalability, and efficient communication between different components of the application.

4. Setup Instructions

Prerequisites

- Node.js
- MongoDB Atlas Account
- Visual Studio Code
- Git

Installation

Clone Repository

```
git clone https://github.com/sowmyakella137-ops/VIP-C2-ShopEZ-E-commerce-application.git
```

Install Frontend Dependencies

```
cd client
```

```
npm install
```

Install Backend Dependencies

```
cd server
```

```
npm install
```

Environment Variables

Create a .env file inside the server folder and configure:

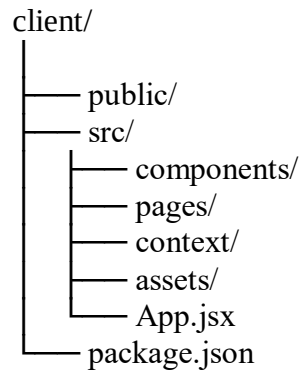
```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

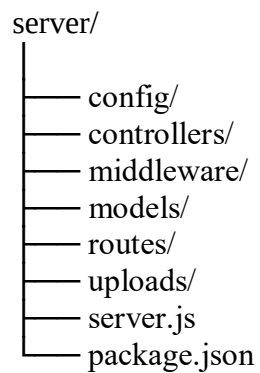
JWT_SECRET=your_secret_key

5. Folder Structure

- **Client:**



- **Server:**



6. Running the Application

Start Frontend

cd client

npm run dev

Start Backend

cd server

npm start

7. API Documentation

Authentication APIs

Register User

POST /api/users/register

Login User

POST /api/users/login

Product APIs

Get All Products

GET /api/products

Get Product by ID

GET /api/products/:id

Add Product (Admin)

POST /api/products

Update Product (Admin)

PUT /api/products/:id

Delete Product (Admin)

DELETE /api/products/:id

8. Authentication

Authentication and authorization in ShopEZ are implemented to ensure that only authorized users can access protected features of the application. The system uses JSON Web Tokens (JWT) along with password encryption to provide secure access control.

User Authentication Process

1. A user registers by providing the required details such as name, email, and password.
2. The password is encrypted using the bcrypt library before being stored in the MongoDB database.
3. When the user logs in, the entered credentials are verified against the stored information.

4. Upon successful authentication, the server generates a JWT token.
5. The token is sent to the client and stored for future requests.
6. For protected routes, the token is included in the request header and verified by the backend before access is granted.

Authorization

The application implements role-based access control to differentiate between regular users and administrators.

- Users can browse products, manage their cart, and place orders.
- Administrators can add, update, and delete products through the admin dashboard.

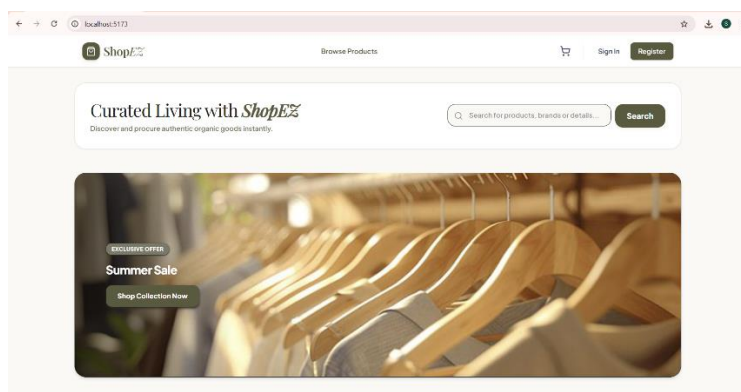
Protected routes ensure that only authenticated users can perform authorized actions.

Security Features

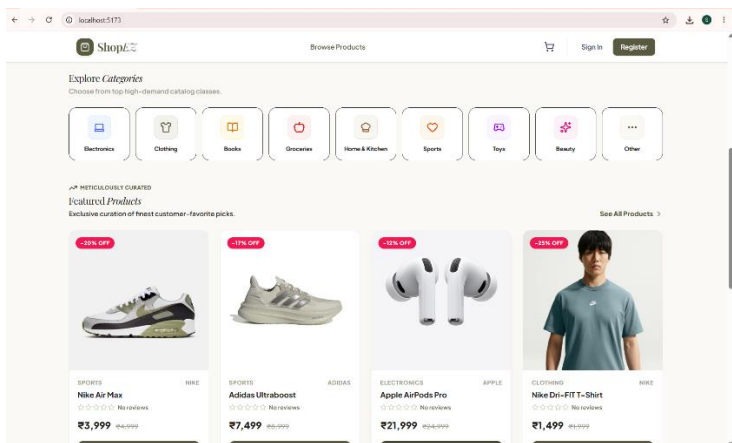
- Password hashing using bcrypt.
- JWT-based authentication for secure user sessions.
- Protected API routes using authentication middleware.
- Validation of user credentials before granting access.
- Secure communication between frontend, backend, and database.

9. User Interface

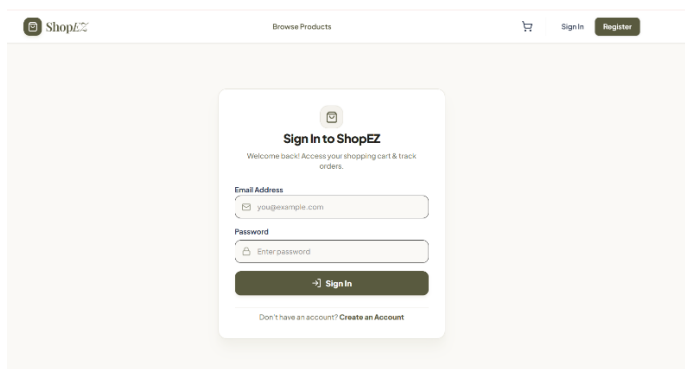
- Home Page



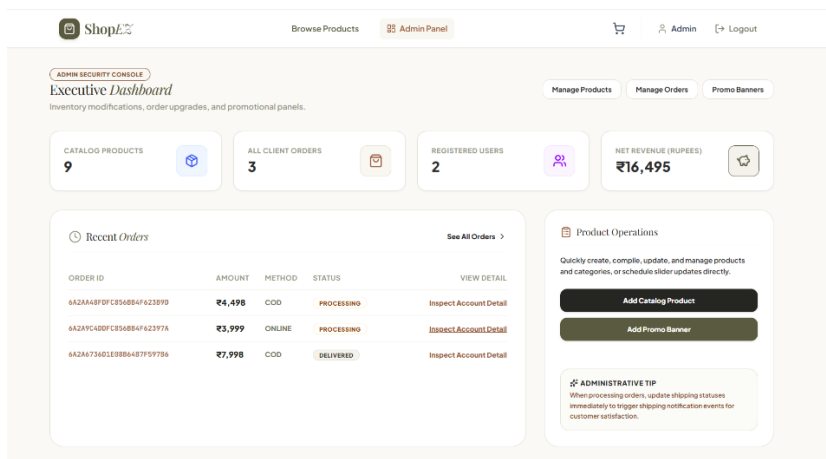
- Product Listing



- Login Page



- Admin Dashboard



10. Testing

Testing was performed to verify that all major functionalities of the ShopEZ application work correctly. The application was tested for user authentication, product management, cart operations, and order placement.

Tools Used:

- Postman
- Google Chrome Developer Tools
- MongoDB Atlas

Result: All core features functioned successfully during testing.

11. Screenshots or Demo

[Project Demo Link](#)

12. Known Issues

The current version of ShopEZ has a few limitations:

- Payment gateway integration is not implemented.
- Advanced product search and filtering features are limited.
- Email notification functionality is not available.

13. Future Enhancements

The following features can be added in future versions of the application:

- Online payment integration.
- Wishlist functionality.
- Product reviews and ratings.
- Advanced search and filtering.
- Order tracking system.
- Email notifications.